



5th Cisco backdoor found

Cisco recently patched an undocumented root password in their Cisco Policy Suite that allows attackers root access. <https://dgit.in/Aug18-52-1>



Suspected car company data breach

A 157 GB file with nearly 47,000 sensitive documents including car blueprints, schematics etc. were out in the open for a while. <https://dgit.in/Aug18-52-2>

*VIDEOS



*Basic usage of Tensorflow

YouTuber Dan Van Boxel, as a part of the Dan Does Data series, goes over the basic usage of Tensorflow library for data flow graphs.

<https://dgit.in/TFBasicUse>



*Effective Tensorflow for Non-Experts

This talk from Google I/O '17 shows how to implement complex machine learning models and deploy them effectively on any platform that supports TensorFlow.

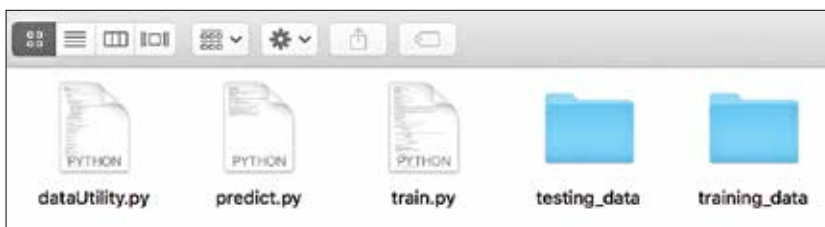
<https://dgit.in/EffTF>



*Introduction to Tensorflow

A lecture from the CS224d Deep Learning for Natural Language Processing course by Richard Socher from Stanford University.

<https://dgit.in/TFIntro>



```
batch(batch_size)
x_valid_batch, y_valid_batch,
_, valid_cls_batch = data.
valid.next_batch(batch_size)
where next_batch is a simple python function in dataUtility.py file that returns the next n images to be passed for training. Similarly, we pass the validation batch of images independently to in another session.run() call. We can calculate the validation accuracy by passing accuracy in session.run() and providing validation images in a feed_dict.
```

After each Epoch which as we know is the single pass of all images in training, we print the accuracy numbers and save the model using saver object.

```
show_progress(epoch, feed_dict_tr, feed_dict_val, val_loss)
saver.save(session, 'n-shapes-model')
```

In summary, the complete training file looks like this: <https://dgit.in/TF8>.

Congratulations! We are done creating our first n image classifier network! This is a tiny network and is not at all good as an image classifier but it will suffice for our learning. Please feel free to run your training and post your accuracies on your channel.

Prediction

Once training completes, we observe the following files in the folder:

```
Training Epoch 1 --- Training Accuracy: 9.4%, Validation Accuracy: 15.6%, Validation Loss: 1.605
Training Epoch 2 --- Training Accuracy: 46.9%, Validation Accuracy: 53.1%, Validation Loss: 1.313
Training Epoch 3 --- Training Accuracy: 68.8%, Validation Accuracy: 78.1%, Validation Loss: 0.729
Training Epoch 4 --- Training Accuracy: 96.9%, Validation Accuracy: 96.9%, Validation Loss: 0.268
```

Output

1. n-images-model.meta
2. n-images-model.data-00000-of-00001
3. n-images-model.index
4. checkpoint

The n-images-model.meta file contains the network graph and we can use this to recreate the graph when required. This is how we do this:

```
saver = tf.train.import_meta_graph('n-shapes-model.meta')
```

The file n-images-model.data-00000-of-00001 contains the weights of the network. As we have recreated the graph let us now restore its weights as follows:

```
saver.restore(sess, tf.train.latest_checkpoint('./'))
```

Next, we need to read & resize the input image to fit our network, and pass it in a feed dict. So, let's do that:

```
x = graph.get_tensor_by_name("x:0")
```

```
y_true = graph.get_tensor_by_name("y_true:0")
```

```
y_test_images = np.zeros((1, len(os.listdir('training_data'))))
```

```
feed_dict_testing = {x: x_batch, y_true: y_test_images}
result=sess.run(y_pred, feed_dict=feed_dict_testing)
```

Finally, we can run a new image of the relevant class using predict script.

```
python predict.py test_square.jpg
```

```
[[ 0.85 0.05 0.05 0.05]]
```

Output contains the probabilities

of the input image being one of the n classes. In this example, probability of the image being a square is higher. 📐

P.S: You can find all the code used in this article at this link: <https://dgit.in/TFCode>.